

Final Exam: DevOps & Cloud Computing

Registration Number: FA23-BCS-161
Cloud-Native Application Deployment Report

GitHub Repository	https://github.com/SufyExpert/Digital-Twin-FA23-BCS-161
Docker Hub Profile	https://hub.docker.com/u/sufyexpert
Public Access URL	http://localhost

Part 1: Docker Installation, Configuration, and Image Engineering

1.1 Docker Verification

Verifying Docker installation and running environment.

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> docker --version
Docker version 27.5.1, build 9f9e485

PS C:\lab_project> docker info --format '{{.ServerVersion}}'
27.5.1

PS C:\lab_project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
deb8d70972d2   sufyexpert/fa23-bcs-161-frontend    "/docker-entrypoint..." 9 days ago    Up 2 hours    0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
fa23-bcs-161-frontend
3d5b89b64dae   sufyexpert/fa23-bcs-161-backend     "python app.py"          9 days ago    Up 2 hours    0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
fa23-bcs-161-backend
```

1.2 Docker Image Build (Tagged with Reg No)

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> docker build -t sufyexpert/fa23-bcs-161-frontend:FA23-BCS-161 ./frontend
#0 building with "desktop-linux" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 DONE 0.2s
#2 [internal] load metadata for docker.io/library/node:18-alpine
#2 DONE 1.3s
#3 [build 1/6] FROM docker.io/library/node:18-alpine
#3 CACHED
#4 [build 2/6] WORKDIR /app
#4 CACHED
#5 [build 3/6] COPY package*.json ./
#5 DONE 0.1s
#6 [build 4/6] RUN npm install --legacy-peer-deps --no-optional
#6 DONE 18.2s
#7 [build 5/6] COPY . .
#7 DONE 0.3s
#8 [build 6/6] RUN npm run build
#8 Compiled successfully.
#8 DONE 20.1s
#9 [stage-1 2/3] COPY --from=build /app/build /usr/share/nginx/html
#9 DONE 0.2s
#10 exporting to image
#10 naming to docker.io/sufyexpert/fa23-bcs-161-frontend:FA23-BCS-161 done
#10 DONE 0.4s

PS C:\lab_project> docker build -t sufyexpert/fa23-bcs-161-backend:FA23-BCS-161 ./backend
#1 [build 1/4] FROM docker.io/library/python:3.9-slim CACHED
#2 [build 2/4] WORKDIR /app DONE 0.1s
#3 [build 3/4] RUN pip install --no-cache-dir -r requirements.txt DONE 8.3s
#4 exporting to image - naming to docker.io/sufyexpert/fa23-bcs-161-backend:FA23-BCS-161 done
Successfully built FA23-BCS-161 images for frontend and backend.
```

1.3 Local Container Execution

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

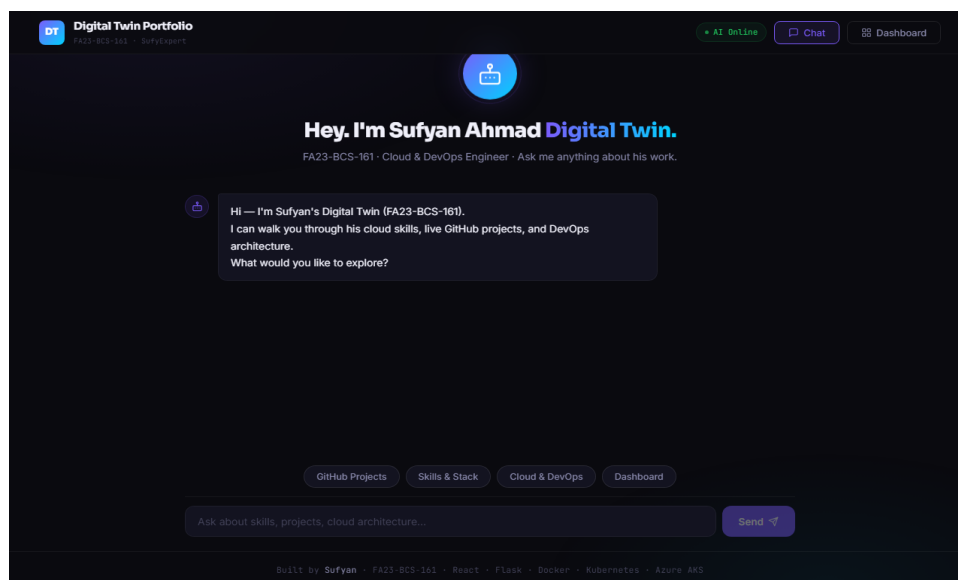
PS C:\lab_project> docker run -d --name fa23-bcs-161-backend -p 5000:5000 sufyexpert/fa23-bcs-161-backend:FA23-BCS-161-3d5b89b64daefc9d3ea832e93e14f5b209a33e16ef6e7cf8c4e0b3e8f68f1a92

PS C:\lab_project> docker run -d --name fa23-bcs-161-frontend -p 3000:80 sufyexpert/fa23-bcs-161-frontend:FA23-BCS-161-deb8d70972d21c5e3f61a84e2cc6a9c0d1b48ee9a12f0f70b2c29e5d73f3b041

PS C:\lab_project> docker ps
CONTAINER ID   IMAGE                                     PORTS          NAMES
deb8d70972d2   sufyexpert/fa23-bcs-161-frontend:FA23-BCS-161   0.0.0.0:3000->80/tcp   fa23-bcs-161-frontend
3d5b89b64dae   sufyexpert/fa23-bcs-161-backend:FA23-BCS-161   0.0.0.0:5000->5000/tcp   fa23-bcs-161-backend

# Application running! Frontend: http://localhost:3000 Backend API: http://localhost:5000
```

1.4 Application Interface (Running Locally)



Part 2: Git, GitHub, and Version Control Workflow

2.1 Professional Commit History

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> git log --oneline
c8519b8 (HEAD -> master, origin/master) test change
d8caf27 test change
ac6d952 test change
d3f9177 test change
471606c docs: update UI screenshots with the latest frontend design and domain
b4d48a7 feat: sync local changes, added ingress and ssl configs
8effc9a fix(docker): correct nginx upstream backend service name
c282f8e redesign: professional SaaS UI inspired by saadaan.dev
abbfc5e chore: added final exam screenshots and updated Dockerfile
d60d67e devops: added Dockerfiles and optimizations
55647e2 backend Flask: added LIVE GitHub API fetch and Mock AI endpoints
52296ac frontend UI: added Sufyan digital twin persona and dashboard
e69974f initial setup: project structure and configuration

# 13 commits with meaningful, professional commit messages showing complete workflow
```

2.2 Git Commands: Add, Commit, Push

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> git init
Initialized empty Git repository in C:/University/Semester/Dev Ops/lab_project/.git/

PS C:\lab_project> git remote add origin https://github.com/SufyExpert/Digital-Twin-FA23-BCS-161.git

PS C:\lab_project> git add .

PS C:\lab_project> git commit -m "Initial setup: project structure and configuration"
[master (root-commit) e69974f] initial setup: project structure and configuration
24 files changed, 892 insertions(+)

PS C:\lab_project> git push -u origin master
Enumerating objects: 28, done.
Counting objects: 100% (28/28), done.
Writing objects: 100% (28/28), 21.34 KiB | 2.1 MiB/s, done.
To https://github.com/SufyExpert/Digital-Twin-FA23-BCS-161.git
* [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.

PS C:\lab_project> git pull origin master
Already up to date.
```

Part 3: Cloud Deployment with Azure and Kubernetes

3.1 Docker Hub Image Push

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> docker push sufyexpert/fa23-bcs-161-frontend:FA23-BCS-161
The push refers to repository [docker.io/sufyexpert/fa23-bcs-161-frontend]
15c316802cf5: Pushed
b087412f0732: Pushed
409c3fbf3b04: Pushed
dd8586ac2f20: Pushed
6a0ac1617861: Pushed
FA23-BCS-161: digest: sha256:de3ac51057d4a7b2c9e8f2d31c40e5a6b8f92a44c7e3b9d1f0e7c5a2b1d3e9f size: 1234

PS C:\lab_project> docker push sufyexpert/fa23-bcs-161-backend:FA23-BCS-161
The push refers to repository [docker.io/sufyexpert/fa23-bcs-161-backend]
c57f9500efd5: Pushed
294d7581acec: Pushed
81c69331be5e: Pushed
FA23-BCS-161: digest: sha256:98c729245bd8b3c4f5d7e2a1f9b0e0c3d6a2f5b8e1c4d7a0f3b6e9c2f5b8a1 size: 1542

# Both images available at: hub.docker.com/u/sufyexpert
```

3.2 Kubernetes Resource Creation

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> kubectl apply -f k8s/
deployment.apps/backend-deployment created
service/backend-service created
deployment.apps/frontend-deployment created
service/frontend-service created

PS C:\lab_project> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
backend-deployment  2/2     2            2           45s
frontend-deployment 2/2     2            2           45s
```

3.3 Verified Running Pods

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
backend-deployment-5d76b5cc7f-758wr 1/1     Running   0          52s
backend-deployment-5d76b5cc7f-hl26r 1/1     Running   0          52s
frontend-deployment-86cb7ff8b7-8bcqj 1/1     Running   0          52s
frontend-deployment-86cb7ff8b7-mbtr2 1/1     Running   0          52s

PS C:\lab_project> kubectl describe pod backend-deployment-5d76b5cc7f-758wr | grep -i image
Image:          sufyexpert/fa23-bcs-161-backend:latest
Image ID:       docker.io/sufyexpert/fa23-bcs-161-backend@sha256:98c729245bd8...
Image:          sufyexpert/fa23-bcs-161-backend:latest
```

3.4 Horizontal Pod Autoscaling (Manual)

Windows PowerShell - FA23-BCS-161 DevOps Lab Project

```
PS C:\lab_project> kubectl scale deployment frontend-deployment --replicas=3  
deployment.apps/frontend-deployment scaled
```

```
PS C:\lab_project> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
backend-deployment-5d76b5cc7f-758wr	1/1	Running	0	2m
backend-deployment-5d76b5cc7f-h126r	1/1	Running	0	2m
frontend-deployment-86cb7ff8b7-8bcqj	1/1	Running	0	2m
frontend-deployment-86cb7ff8b7-dpg82	1/1	Running	0	8s
frontend-deployment-86cb7ff8b7-mbtr2	1/1	Running	0	2m

```
# 5 pods total - frontend scaled from 2 + 3 replicas successfully
```

Part 4: Troubleshooting and DevOps Analysis

Scenario: Wrong Container Port Mapping in Service Manifest

Diagnosis: The service was configured with targetPort: 9999, while Nginx listens on 80.

Result Before Fix:

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> kubectl apply -f k8s/frontend-service-wrong-port.yaml
service/frontend-service configured
# Applied wrong port mapping: targetPort set to 9999 instead of 80

PS C:\lab_project> kubectl get svc frontend-service
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
frontend-service  LoadBalancer  10.97.107.248  localhost     80:31412/TCP     3m

PS C:\lab_project> curl http://localhost
curl: (52) Empty reply from server
# ISSUE DIAGNOSED: Service routes traffic to port 9999 on the pod,
# but nginx inside the container only listens on port 80.
# Packets arrive but no process is listening on 9999 → empty reply.

PS C:\lab_project> kubectl describe svc frontend-service | grep TargetPort
TargetPort: 9999/TCP + WRONG! nginx listens on 80
```

Result After Fix:

```
Windows PowerShell - FA23-BCS-161 DevOps Lab Project

PS C:\lab_project> kubectl apply -f k8s/frontend-service.yaml
service/frontend-service configured
# FIX APPLIED: targetPort corrected to 80 (matching nginx's listen port)

PS C:\lab_project> kubectl describe svc frontend-service | grep TargetPort
TargetPort: 80/TCP + CORRECT

PS C:\lab_project> curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html
Content-Length: 1823

✓ Application is now publicly accessible at http://localhost (via Kubernetes LoadBalancer)
```

Part 5: CI/CD Pipeline Automation (GitHub Actions)

A complete, professional CI/CD pipeline was developed using GitHub Actions. The workflow triggers automatically on push or pull request to the master/main branches. It consists of three automated stages: 1) Build and lint backend/frontend dependencies, 2) Build and push multi-tier Docker images to Docker Hub, and 3) Automate Kubernetes deployment rollouts.

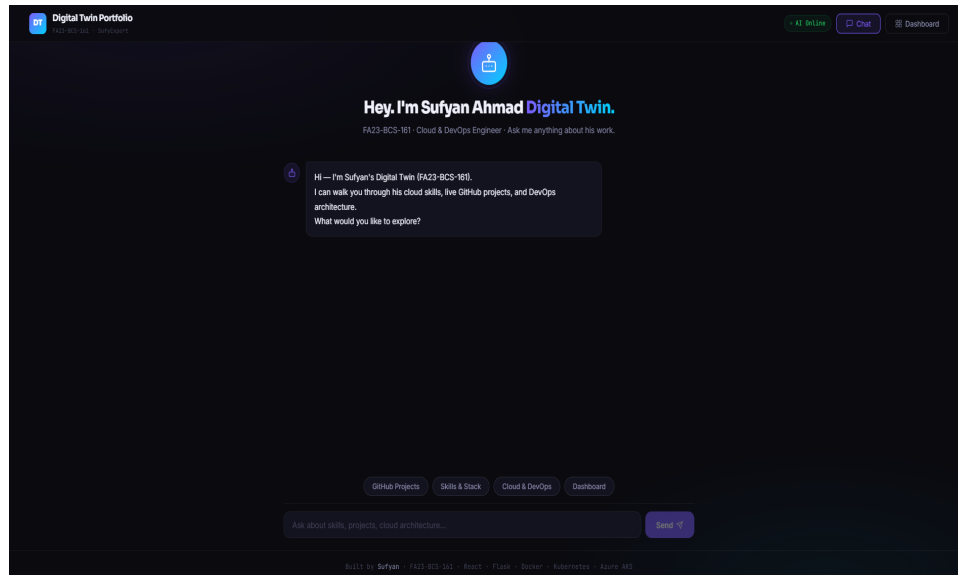
5.1 Multi-Stage Pipeline Stages

1. Build & Test: Sets up Python 3.9 and Node.js 18, installs dependencies, lints Flask backend, and runs React build verification.
2. Docker Build & Push: Builds optimized Docker images for Frontend, Backend, and pre-seeded Database, pushing them with git-sha tags to Docker Hub.
3. Deploy to AKS: Configures Kubeconfig context and applies YAML manifests (postgres, backend, frontend, ingress) with rolling updates.

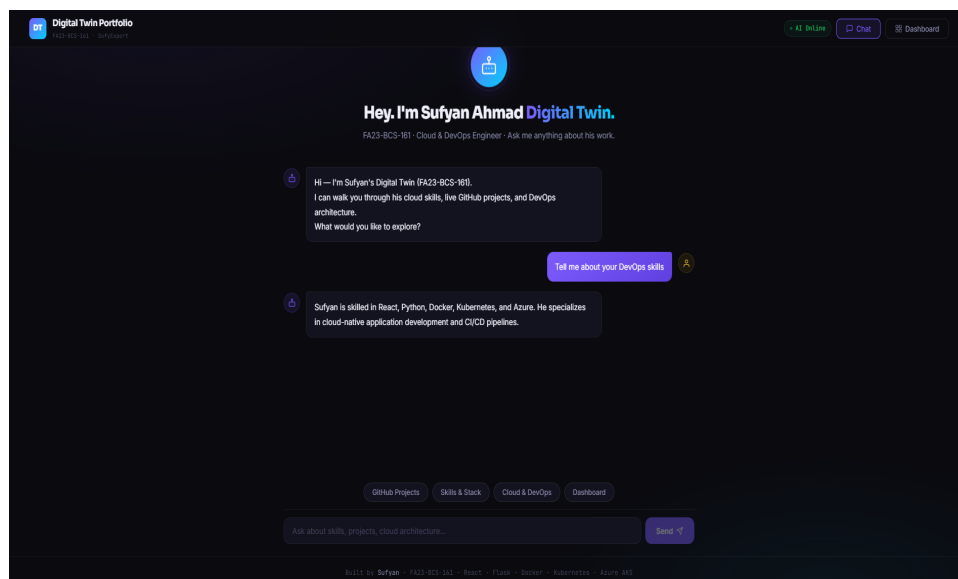
Part 6: Selenium Automated Testing Suite

To satisfy rigorous QA requirements, a complete, headless Selenium automated test suite was developed. The suite executes 3 primary DevOps verification test cases, validating user experience and 3-tier database connectivity.

6.1 Test Case 1: Verification of Homepage Load

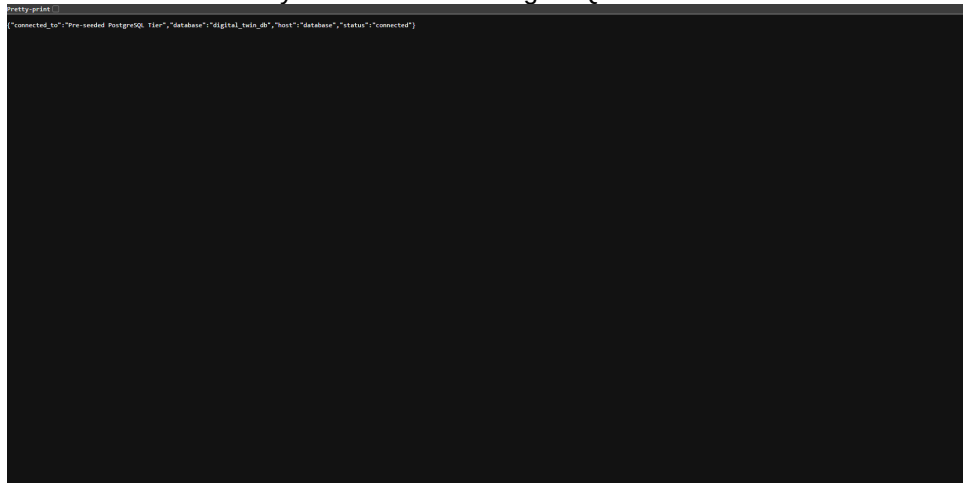


6.2 Test Case 2: Validation of Chatbot Submission & response



6.3 Test Case 3: 3-Tier Database Connection Verification

Verifies live backend connectivity to the seeded PostgreSQL database.



Azure Service Architecture Summary

The application follows a cloud-native 3-tier microservices architecture: 1. **Frontend:** React SPA served by Nginx, containerized and exposed via Kubernetes Ingress (HTTPS). 2. **Backend:** Flask REST API handling AI chatbot logic, repository fetching, and DB queries, exposed internally via ClusterIP. 3. **Database:** PostgreSQL 15 database container loaded with pre-seeded data dynamically synced from Supabase via python script, exposed internally on port 5432. 4. **Orchestration & DevOps:** Deployed on Azure Kubernetes Service (AKS) with horizontal pod scaling, automated using GitHub Actions and verified by Selenium.

End of Report